IIT MADRAS · SOFTWARE ENGINEERING · SEPT 2026

Milestone 2 Report

Scheduling, Design and Frontend

Student Mentoring and Career Support Platform

Problem Statement 2

Team 032 – Himalayan Coders

Himanshu Rathor · Ankur Maurya · Hemendra Kumar Sharma
Adnan Mustafa · Bojannagari Pavan Kumar Reddy

1. Project Schedule and Sprint Plan

1.1 Scheduling tool

We use **Jira** (project key KAN) to plan and track our work. An epic groups the work of one milestone or sprint. Each task has one owner, a start date and a due date, and moves across the board columns: To Do, In Progress, In Review and Done. The Jira **Timeline** is our Gantt chart and the Jira **Board** is our Kanban board. Because every task has an owner, both views show what each member does.

1.2 Team roles

Member	Roles
Himanshu Rathor	Scrum Master, Product Manager
Ankur Maurya	Backend Developer, Frontend Developer
Hemendra Kumar Sharma	Frontend Developer, Backend Developer
Adnan Mustafa	Tester, Code Reviewer
Bojannagari Pavan Kumar Reddy	Code Reviewer, Tester

1.3 Overall schedule

Phase	Dates	Focus
Milestone 1	until 25 Oct	User requirements
Milestone 2	26 Oct – 1 Nov	Scheduling, design, frontend pages
Sprint 1 (Milestone 3)	2 Nov – 22 Nov	APIs, test cases and user testing for the core flow
Sprint 2 (Milestone 4)	23 Nov – 6 Dec	APIs, test cases and user testing for the rest of the flow
Milestone 5	7 Dec – 20 Dec	Final report, demo video, working prototype

1.4 Sprint 1 (2 Nov – 22 Nov): core client-mediated flow

Jira task	User stories	Dates
Registration and login APIs	US-S01, US-M01	2 – 8 Nov
Student and mentor profile APIs	US-S02, US-M02, US-C05	6 – 12 Nov
Admin dashboard and role-based access APIs	US-C01, US-C06	9 – 15 Nov
Match creation APIs	US-C02	12 – 18 Nov
Test cases, pytest and Swagger YAML	all of the above	9 – 20 Nov
User feedback session and Sprint 2 plan	–	20 – 22 Nov

1.5 Sprint 2 (23 Nov – 6 Dec): confirmation, tracking and student views

Jira task	User stories	Dates
Match confirmation and notification APIs	US-C03	23 – 29 Nov
Student goals, preferences and availability APIs	US-S03 to US-S06	23 – 29 Nov
Match status management and overview APIs	US-C04, US-C07	26 Nov – 2 Dec
Student connection view, readiness, concern, feedback and status APIs	US-S07 to US-S11	28 Nov – 3 Dec
Smart Match AI agent API (score and reason, rule-score fallback)	US-C02 (extension)	by 2 Dec
Mentorship Pulse check-in APIs and weekly reminder emails	US-S09, US-S10 (extension)	by 3 Dec
Mentorship Pulse AI review and admin review queue	US-C04 (extension)	by 4 Dec
Test cases, pytest and Swagger YAML	all of the above	30 Nov – 5 Dec
User feedback session and final-phase plan	–	4 – 6 Dec

All 21 user stories from Milestone 1 are covered across the two sprints.

Planned extension. Two AI features are designed in Milestone 2: **Smart Match** (an AI agent scores student and mentor pairs from what they wrote) and **Mentorship Pulse** (weekly check-ins with reminder emails and an AI review that sends only the cases that need attention to the admin). They build on US-C02, US-C03, US-C04, US-S09 and US-S10. They are built in Sprint 2, after the core flow, as the three AI tasks in the table above (Jira KAN-40 to KAN-42).

1.6 Iterations

Each sprint is split into weekly iterations. Every sprint ends with a user feedback session, and its findings shape the next sprint. Code Reviewers review each task and Testers test it before it moves to Done.

1.7 Gantt chart

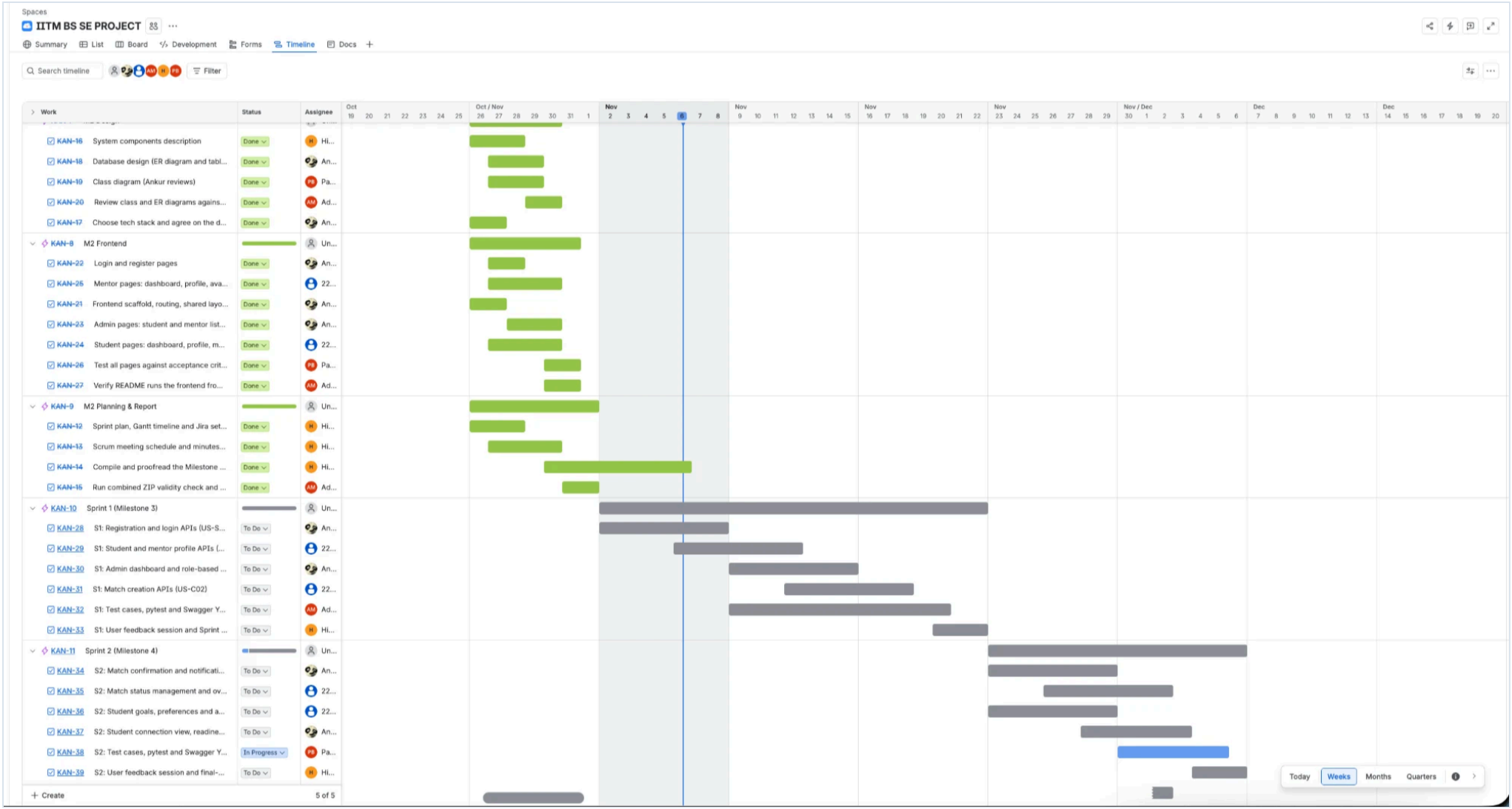


Figure A. Jira Timeline (Gantt) with the epics expanded, from 26 Oct to December.

1.8 Kanban board

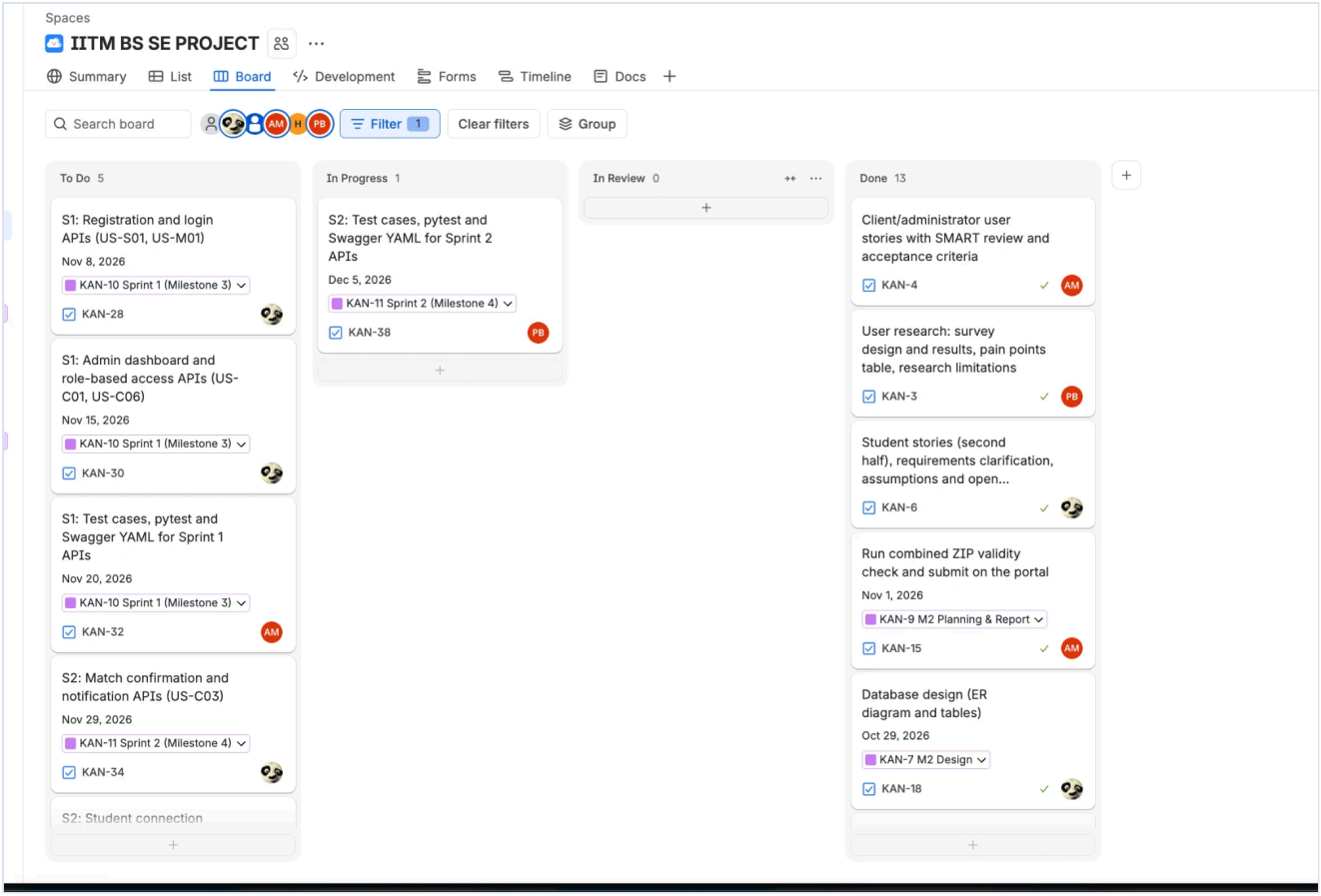


Figure B. Jira Kanban board (filtered to the team members): all Milestone 2 tasks are in Done; the Sprint 1 and Sprint 2 tasks are still to do.

2. Scrum Meetings Schedule and Minutes

Scrum meetings: held about twice a week on Google Meet and led by the Scrum Master. Each meeting covers what was done, what comes next and any blockers. Three meetings were held in Milestone 2.

Minutes from Scrum Meeting 1

Date and time	27 October 2026, 6:00 – 6:57
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra, Himanshu, Pavan
Agenda	Frontend design; database schema; workflow for matching students to mentors.

Discussion. The team discussed the overall design of the frontend and the database schema of the platform. The team also discussed how a student is matched to a mentor. Each match will show a status of Proposed, Active, Completed or Cancelled.

Decision. The match status values are fixed as Proposed, Active, Completed and Cancelled.

Next meeting agenda. Continue discussing the frontend design in more detail, and decide how the dashboards will look for the Admin, the Student and the Mentor.

Minutes from Scrum Meeting 2

Date and time	31 October 2026, 7:00 – 7:45 PM
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra
Agenda	Scrum call 2 minutes of meeting.

#	Discussed	Action (owner)
1	Jira tasks and dashboard.	The team reviews their tasks and makes changes.
2	Changes in the ER diagram.	Ankur (completed)
3	Milestone 2 report review.	Adnan
4	User story US-C03 clarification: the design was modified as per this story.	Ankur (completed)
5	GitHub Milestone 2 review.	The team, as per the tasks assigned in Jira

Minutes from Scrum Meeting 3

Date and time	3 November 2026, 9:00 – 9:50
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra, Himanshu, Pavan
Agenda	Mentor-student relationship tracking; AI features for matching and feedback analysis; admin dashboard UI changes.

Discussion. The team discussed how to track the mentor-student relationship over time. The team also discussed adding AI capabilities: recommending student-mentor matches to the admin, and automatically analysing the sentiment of feedback, with the results shown to the admin. The changes to the admin overview dashboard were discussed too, including a feedback form and the relationship status in the UI.

Decisions. (1) A feedback form will be emailed periodically to both the mentor and the student to track the relationship, either every week or every 3 days. (2) The admin dashboard UI will be updated to include a feedback form and the relationship status.

Next meeting agenda. Review all the changes discussed in this meeting and go over them in detail.

3. Design of Components

Our platform is **client-mediated**. Students and mentors each keep their own profile, but they cannot see each other. The admin (our client) looks at both sides, proposes a match and follows it until it ends.

Component	What it does
Login and roles	Registration and login for three roles: student, mentor and admin. Each role sees only its own pages.
Student module	The student keeps their goals, skills, wanted topics, preferred format, duration and availability, and writes in a free-text box what they need from a mentor. The student sees the status of their match, accepts or declines the mentor proposed by the admin, answers a short check-in every week, gives feedback and can raise a concern.
Mentor module	The mentor keeps their qualifications, topics, bio, availability slots and the maximum number of students they can mentor at the same time. The mentor writes in a free-text box what they can offer. The mentor accepts or declines match requests, sees their active mentees and answers a weekly check-in for each of them.
Admin module	The admin views student profiles, reviews and verifies mentor profiles, sees recommended matches, proposes a match, confirms or cancels it, changes its status, reviews the weekly check-ins that the AI flagged, and follows concerns and feedback.
Matching and recommendation (Smart Match)	An AI agent gives each student and mentor pair a score out of 100 with a reason, and the admin sees the best mentors first. It only suggests. The admin decides. See section 3.1.
Mentorship Pulse	Weekly check-ins for both people in an active match, reminder emails, and an AI review that flags the answers that need a person. See section 3.2.
Match workflow	Controls the status of a match: proposed, active, completed or cancelled. A match becomes active only when the admin confirms it after both sides have accepted. Then each person receives the other's email address, and the weekly check-ins start. Every change is saved in a history.
Notifications and email	Tells the mentor and the student when a match is proposed. Tells the admin when someone accepts or declines, and when a check-in is flagged. Tells both people when the admin confirms or cancels. Emails go out when a match becomes active (contact details) and when a weekly check-in is due (reminder).
Database	Stores users, profiles, topics, mentor availability, matches with their scores, status history, weekly check-ins, AI reviews, feedback, concerns and notifications.

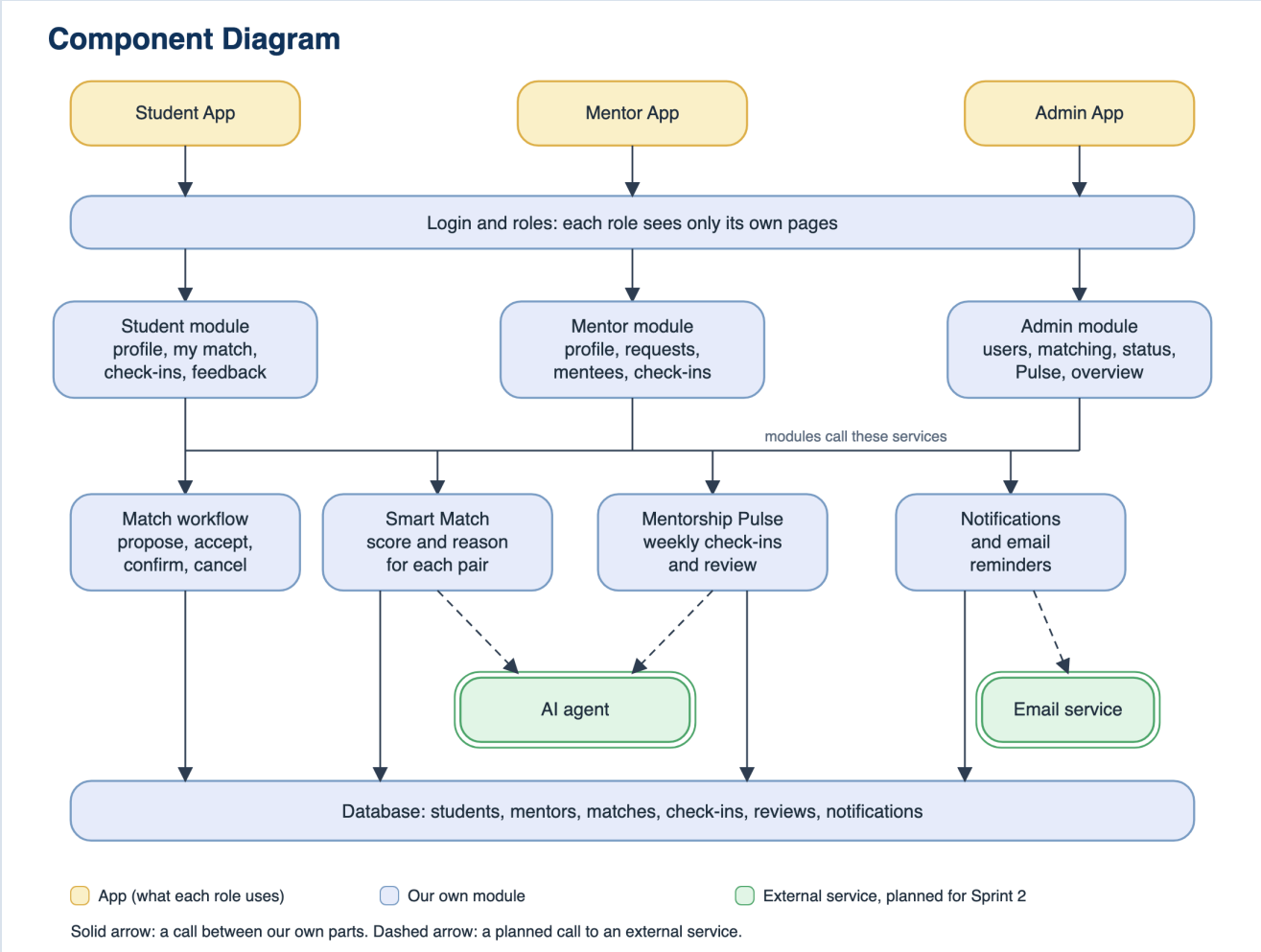


Figure C. Component diagram. Each role has its own app and module behind one login. The modules call four services that share one database; the AI agent and the email service (green) are planned for Sprint 2.

How a match moves: the admin proposes a match, and the mentor and the student are both notified. Each of them accepts or declines. When **both have accepted**, the admin can confirm the match and it becomes active. The admin can also cancel it at any time. The two people then write to each other and agree privately on session times and how they will talk. Later an active match is completed or cancelled. If someone declines, the admin cancels the match and can propose a new one.

3.1 How matches are recommended (Smart Match)

The admin should not have to compare every student with every mentor. The platform makes a short list in three steps.

Step	What happens
1. Free text	The student writes what they need from a mentor, and the mentor writes what they can offer, in their own words (up to 800 characters). Both still choose topics from the same list.
2. Fixed rules	Only allowed pairs are scored: the mentor is verified, the mentor has a free place (the maximum set by the mentor, 1 to 10, default 3, counts proposed and active matches), and the student has no open match.

3. AI score	An AI agent on the server reads the two texts, the topics and the mentor's years of experience, and returns a **score from 0 to 100 and a short reason** for each allowed pair. A score under 40 is shown as a weak fit.

The admin still decides. The agent only recommends. It never proposes, confirms or cancels. If the agent is unavailable, the platform uses the earlier **rule score** (topic overlap 70 + mentor experience 30, years counted up to 10), so the admin always has a list. The rule score is also shown next to the AI score for comparison.

Privacy. Only the two texts, the topics and the years of experience are sent to the agent. Names and email addresses are not. The profile form tells people that Smart Match reads what they write and asks them not to add phone numbers or addresses.

Example. Neha wrote that she wants to become a product manager and needs help with product requirements and interviews. Priya wrote that she mentors on product requirements, roadmaps and mock interviews. The reason shown to the admin is: "Both describe: product, manager, writing, requirements, interviews, changing; covers 2 of 2 wanted topics; 8 years of experience", with a score of **98** (rule score 94). Neha with Dr. Meera, who coaches data careers, scores **38** and is shown as a weak fit.

In the prototype the pages show this result from a simple offline stand-in that compares the wording of the two texts. It has the same inputs and outputs as the agent, so the pages do not change when the agent is connected in Sprint 2. The screens carry a note saying so.

3.2 Mentorship Pulse: weekly check-ins with human review

After a match becomes active, the platform keeps track of how it is going without the admin having to ask.

Step	What happens
1. Contact	Both people receive an email with the other person's address. They agree session times and how they will talk (call, video, messages) between themselves. The platform does not run the sessions.
2. Schedule	One check-in is created for each week of the mentorship, for the student and for the mentor. The number of weeks follows the chosen duration: 1 month = 4 weeks, 3 months = 12, 6 months = 24 (4 if not sure).
3. Reminder	When a week ends, each person gets an email with a link to their Check-ins page.
4. Answer	A short form: a rating from 1 to 5 and an optional comment. The answers go to the admin only, not to the other person.
5. AI review	An AI agent reads each answer and gives a verdict (**on track, watch or needs attention**) with a short reason. It looks at the rating and the words in the comment. A rating of 2 or less is always flagged. A week without a session is not a problem by itself, because both people may agree to skip it; if something is wrong, they say so in the rating or the comment.
6. Human in the loop	An answer that is "on track" is only saved. "Watch" and "needs attention" are sent to the admin, who reads the reason, writes a note and marks it reviewed. A check-in that is still unanswered 2 days after it was due is also sent to the admin.

The agent never changes a match, cancels it or contacts anyone. It only decides what a person should look at, which keeps the admin's work small. The admin sees all of this on one page, **Mentorship Pulse**: what needs review, the active mentorships with their latest verdict, and the emails sent. This is how the admin follows the condition of every mentorship (US-C04), together with student feedback (US-S10) and concerns (US-S09).

In the prototype the emails are recorded in a list instead of being sent. The verdicts come from a rule-based stand-in with the same output as the agent.

Admin dashboard. The overview is kept simple. A thin line at the top shows the number of matches in each status and the number of students and mentors. The first card, "Needs your attention", lists seven things on one line each, with a count and a button that goes to the right page: matches ready to confirm, proposals declined by one side, mentors waiting to be verified, students waiting for a match, proposals waiting more than 3 days, weekly check-ins flagged for review, and open concerns. The second card shows the best mentor for each waiting student, with the Smart Match score and reason and a Propose button. The last card has four tabs, shown one at a time: open concerns, student feedback, mentor capacity and recent activity. Everything the admin needs is on this one page, and nothing is hidden. When a pair is proposed, its score and reasons are saved with the match. The server version of this logic is built in Sprint 1 (match creation APIs).

4. Class Diagram

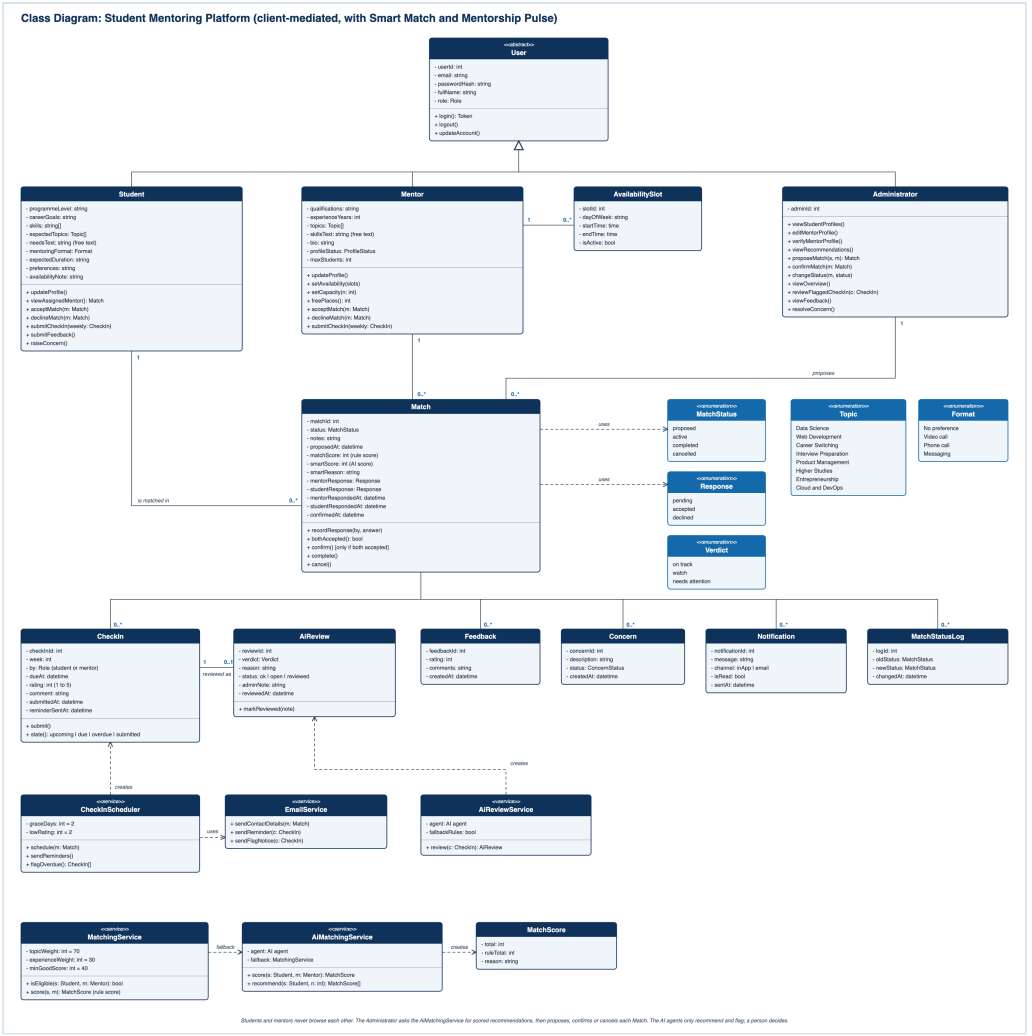


Figure 1. Class diagram. Student, Mentor and Administrator are kinds of User. Every Match is proposed by an Administrator and links one Student and one Mentor. The AiMatchingService scores pairs (with the MatchingService rule score as fallback). The CheckInScheduler creates the weekly CheckIns and the AiReviewService reviews them.

5. Database Design

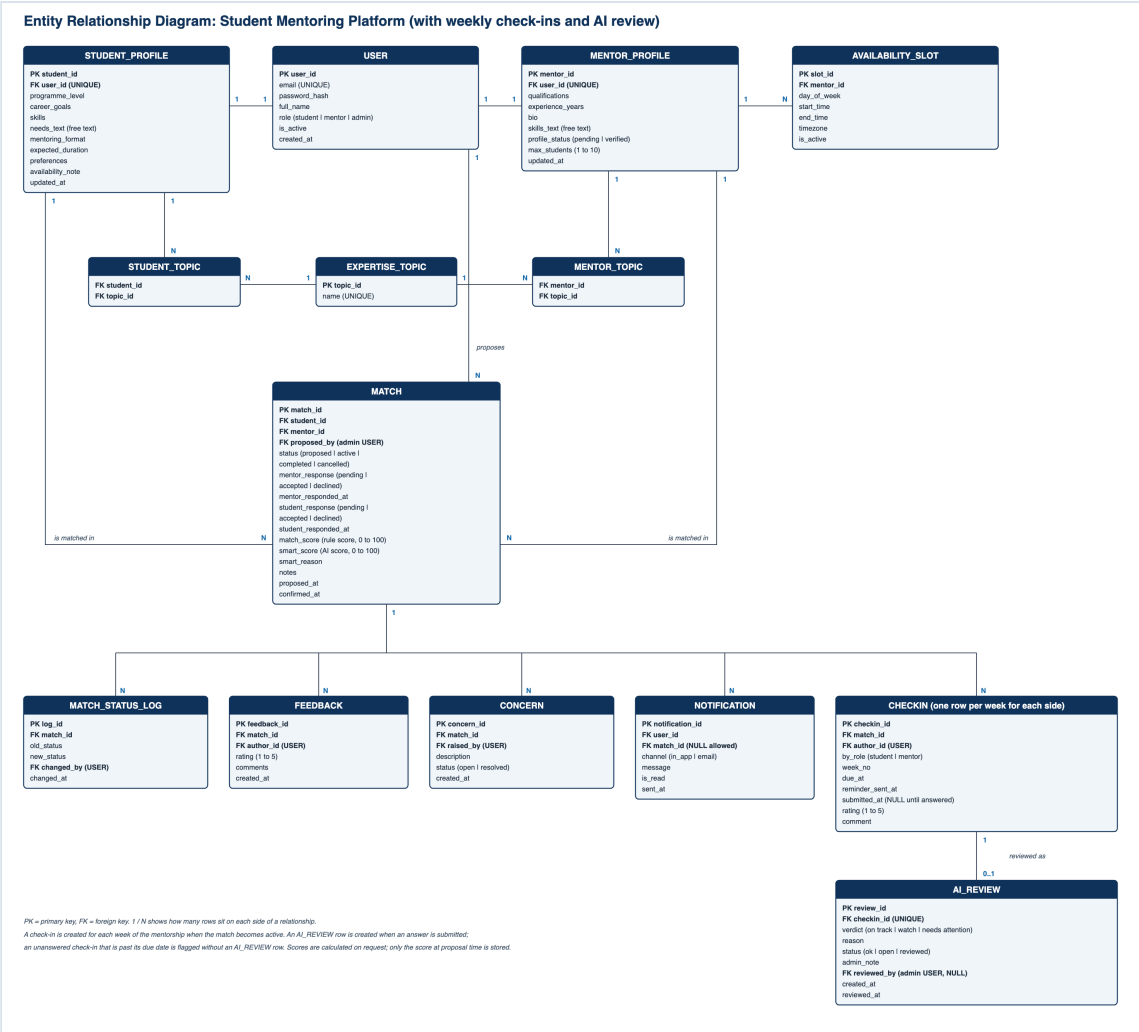


Figure 2. Entity relationship diagram.

Table	What it stores
USER	One row for each account. The email is unique. The role is student, mentor or admin.
STUDENT_PROFILE	Goals, skills, preferred format and duration, preferences, availability and the free text about what the student needs (`needs_text`). One per student.
MENTOR_PROFILE	Qualifications, experience, bio, the free text about what the mentor offers (`skills_text`), verification status and the maximum number of students (`max_students`). One per mentor.
AVAILABILITY_SLOT	The weekly time slots of a mentor. A mentor can have many.
EXPERTISE_TOPIC	The shared list of topics, for example Data Science or Web Development.
STUDENT_TOPIC, MENTOR_TOPIC	Which topics a student wants and which topics a mentor covers.
MATCH	One student and one mentor. It stores the admin who proposed it, the status, the mentor's and the student's answers with dates, the rule score and the AI score (`smart_score`) with its reason, and the confirmation date.
MATCH_STATUS_LOG	The history of every status change: who changed it and when.
CHECKIN	One row for each week and each side of an active match: due date, reminder date, and, once answered, the rating and the comment.
AI_REVIEW	The verdict of the AI agent for one answered check-in (on track, watch, needs attention), its reason, whether it is still open, and the admin's note and name when reviewed.
FEEDBACK	The rating and comments of a student about a match.
CONCERN	A problem raised about a match. It stays open until the admin resolves it.
NOTIFICATION	Messages for users, sometimes linked to a match. The channel is in-app or email.

6. Flows and Backend Design

6.1 Smart Match flow

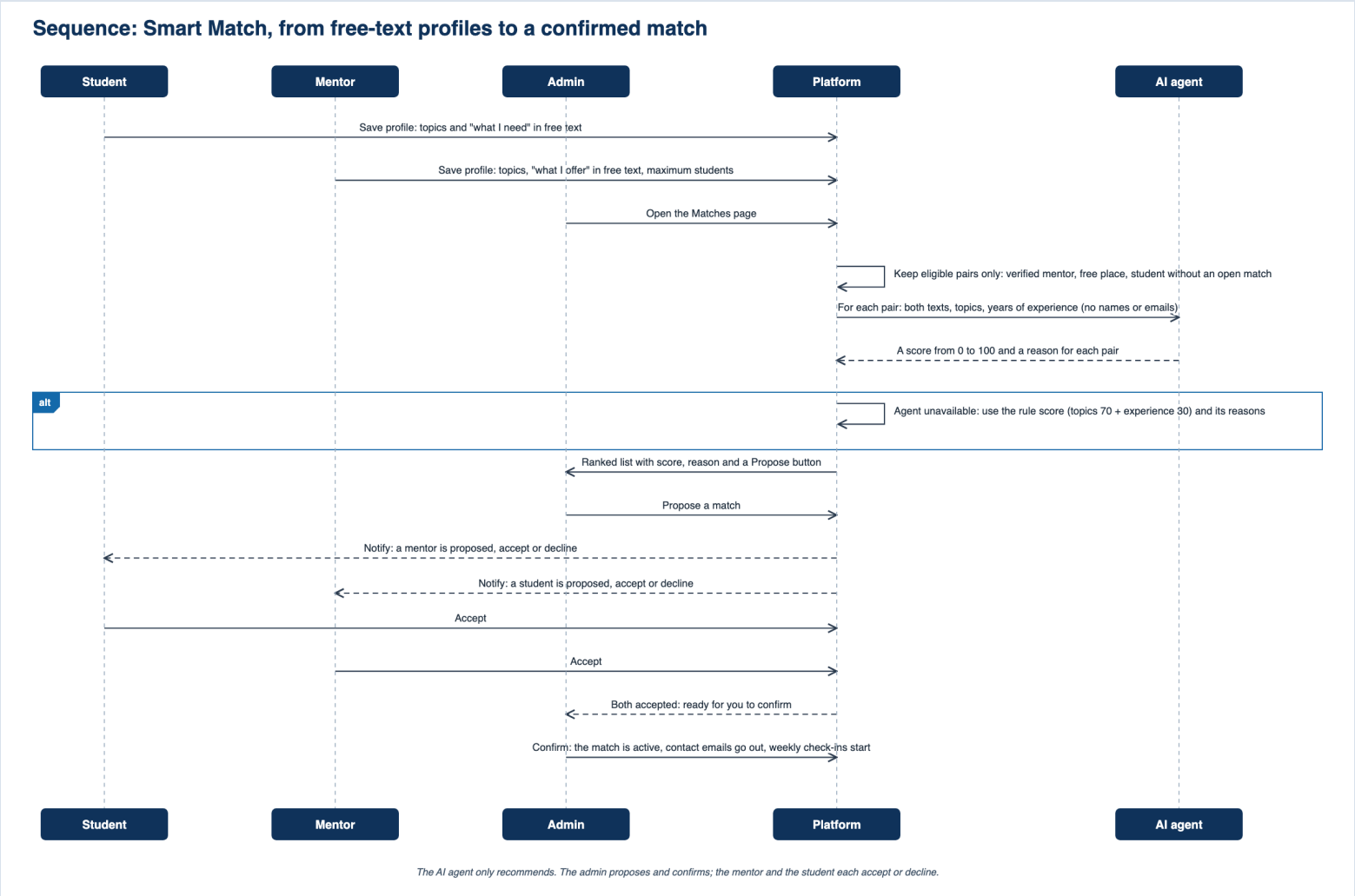


Figure 3. Sequence diagram of Smart Match. The AI agent returns a score and a reason; the admin proposes, and the mentor and student each accept before the admin confirms.

6.2 Weekly check-in flow

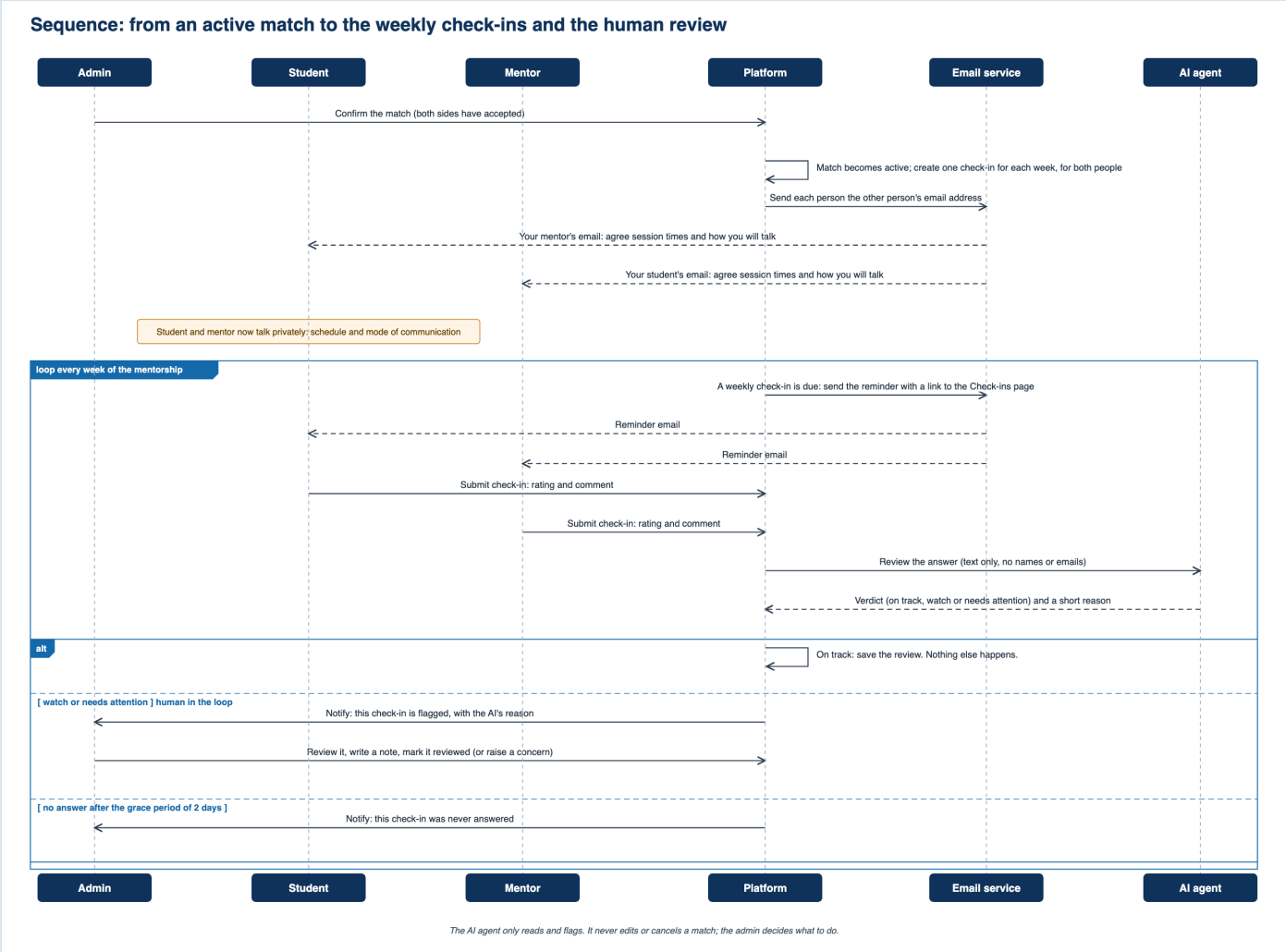


Figure 4. Sequence diagram of Mentorship Pulse. Contact emails, weekly reminders, the AI review and the human review of flagged check-ins.

6.3 How the AI agents are called

- The agents run **on the server**. The browser never holds the AI key; the key is stored as a secret setting of the backend.
- Each agent gets a short instruction and a small JSON input, and must return a small JSON answer. Smart Match returns `{score: 0–100, reason}`; the review returns `{verdict, reason}`. The server checks the answer (score is a whole number from 0 to 100, verdict is one of the three values) and discards anything else.
- If the agent is slow, unavailable or returns an invalid answer, the platform falls back to the rule score for matching, and to the plain rules (rating of 2 or less, or an unanswered check-in) for the review. The admin is never blocked.
- Names and email addresses are never sent. The agent has no way to change data; the server applies the result.
- A scheduled job runs once a day. It creates nothing new; it finds check-ins that have become due, sends the reminder emails once, and flags check-ins that are more than 2 days overdue.

6.4 Planned API endpoints

Endpoint	Used by	Purpose
PUT /students/me/profile	student	Save profile, topics and the free text about needs
PUT /mentors/me/profile	mentor	Save profile, topics, free text about the offer and the maximum students
GET /admin/recommendations?studentId=	admin	Allowed mentors with an AI score and reason (rule score as fallback)
POST /admin/matches	admin	Propose a match; saves the scores and notifies both people
POST /matches/{id}/response	student, mentor	Accept or decline a proposal
POST /admin/matches/{id}/confirm	admin	Only when both accepted: match becomes active, contact emails are sent, check-ins are created
PATCH /admin/matches/{id}/status	admin	Cancel or complete a match
GET /me/checkins	student, mentor	Own weekly check-ins and their state
POST /checkins/{id}/answer	student, mentor	Submit a check-in (rating and comment); the AI review runs and the admin is notified if flagged
GET /admin/pulse	admin	Check-ins to review and active mentorships
POST /admin/reviews/{id}/reviewed	admin	Mark a flagged check-in as reviewed, with a note

The other endpoints (login, profiles, feedback, concerns, notifications) follow the same style and are checked against the caller's role (US-C06). The full list goes in a Swagger YAML file.

7. Frontend Pages and Screenshots

The frontend is a **Vue 3** app (built with Vite) with 14 linked pages. Pages are connected with Vue Router, and each role is redirected to its own pages. Sample data is kept in the browser, so the whole flow can be tried: the admin proposes a match, both sides accept, the admin confirms, and the weekly check-ins start. This milestone is a design, so there is no backend yet and the AI parts show an offline stand-in (see sections 3.1 and 3.2). The data calls are in one folder (`src/services`), so in Milestones 3 and 4 the real APIs replace them and the pages stay as they are.

Page	Used by	Purpose
login, register	everyone	Log in and create an account
student	student	Dashboard with match status and notifications
student-profile	student	Goals, skills, wanted topics, format, duration, availability
student-mentor	student	Status, next step, proposed mentor to accept or decline, feedback, concerns
student-checkins	student	Weekly check-ins: state of each week, the form for the week that is due, the mentor's email
mentor	mentor	Accept or decline requests, see active mentees
mentor-checkins	mentor	Weekly check-ins for each active mentee
mentor-profile	mentor	Background, topics, maximum students, availability
admin	admin	Things that need attention, recommended matches, and tabs for concerns, feedback, capacity and activity
admin-students	admin	View, search and filter student profiles
admin-mentors	admin	Review, edit and verify mentors, availability and capacity
admin-matches	admin	Recommended matches with Smart Match scores and reasons, propose, confirm or cancel, change status
admin-pulse	admin	Mentorship Pulse: check-ins to review, active mentorships, emails sent

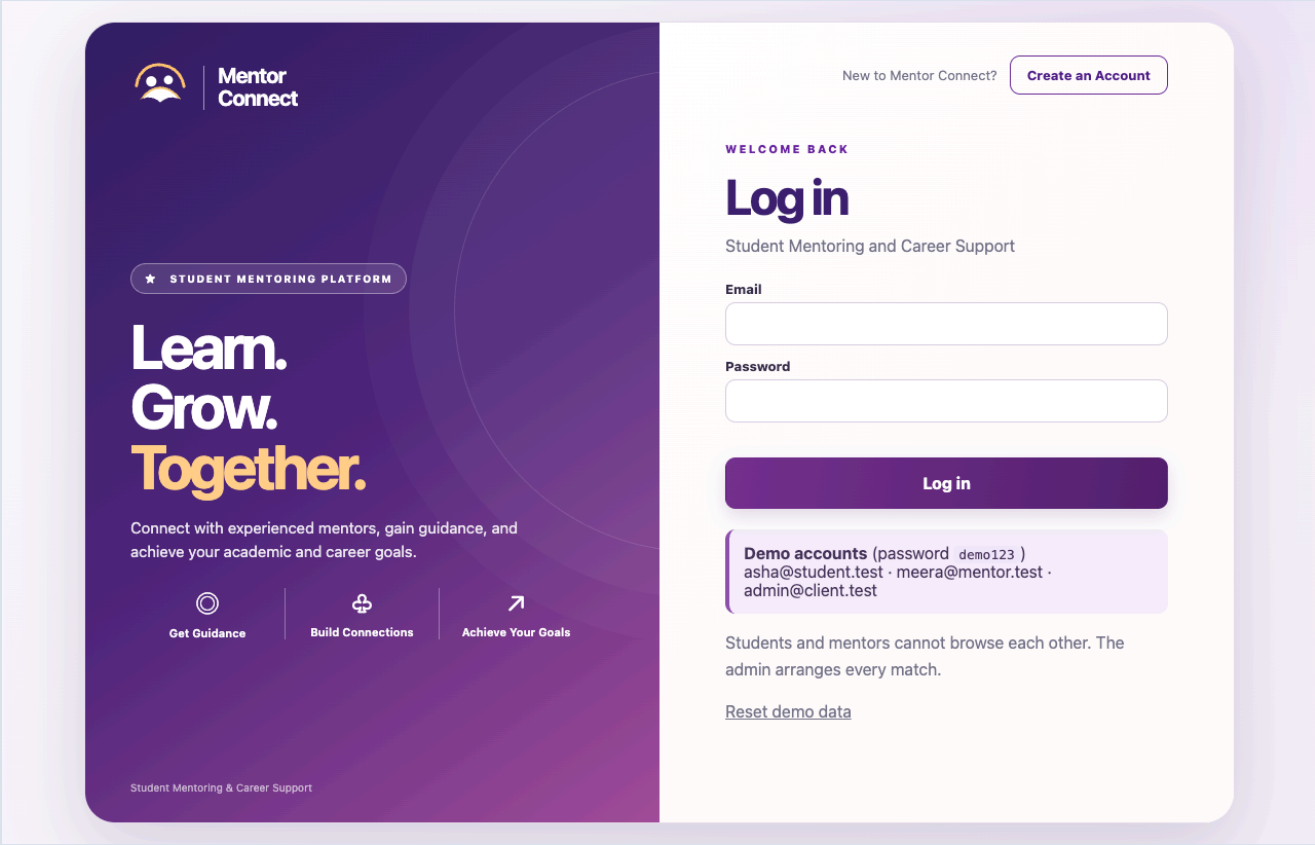


Figure 5. Login page with demo accounts.

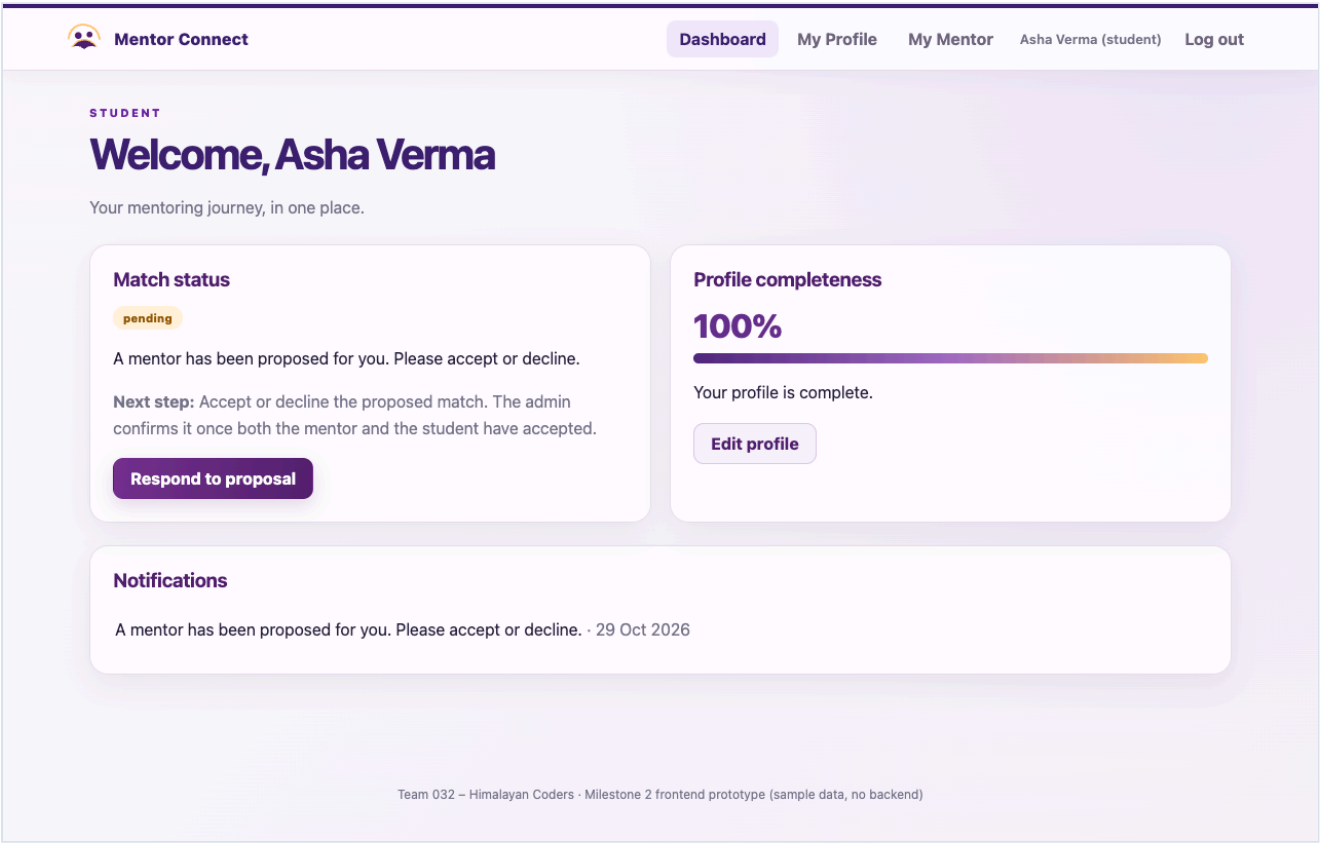


Figure 6. Student dashboard.

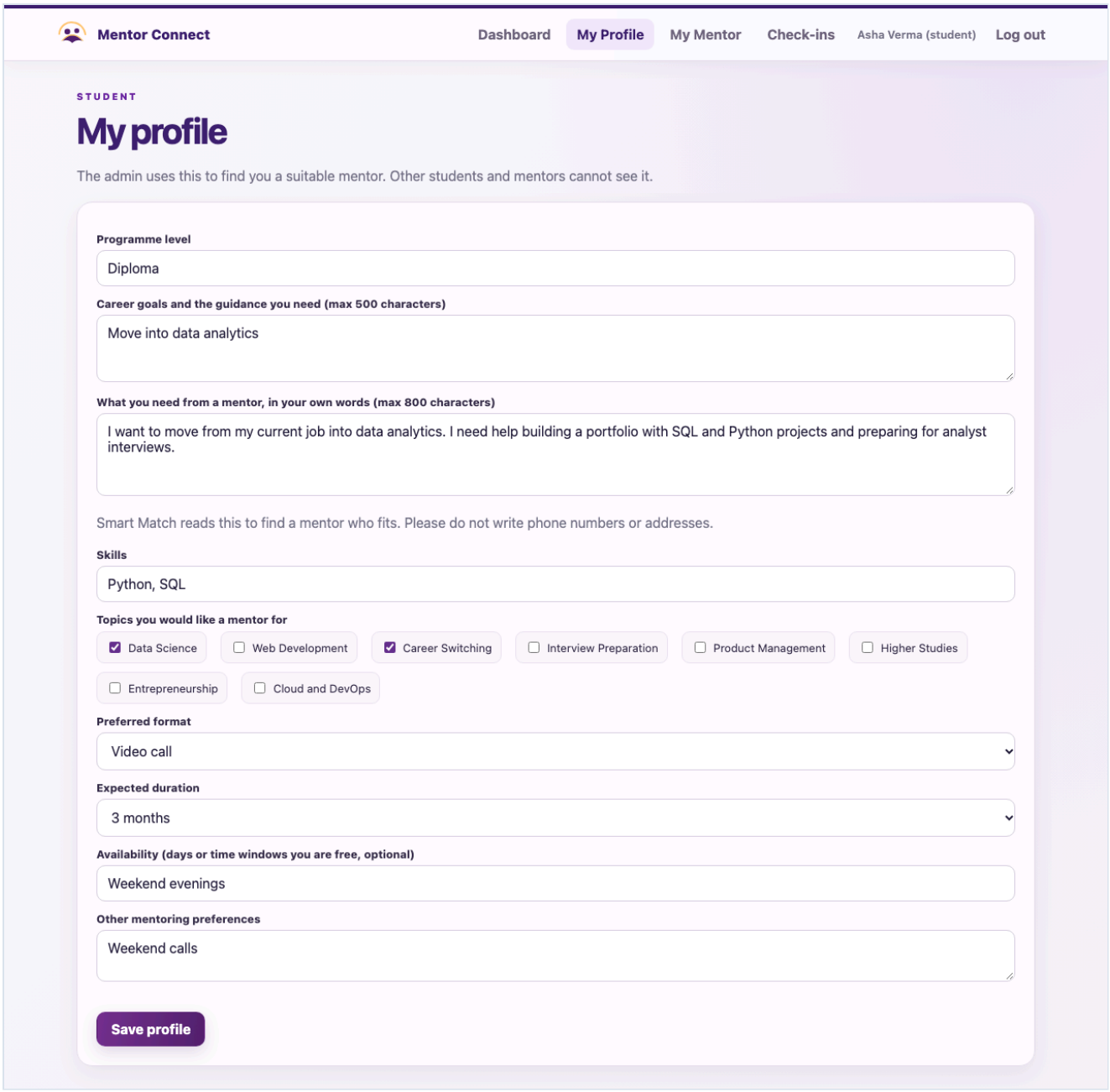


Figure 7. Student profile with wanted topics, format, duration, availability and the free-text box for Smart Match.

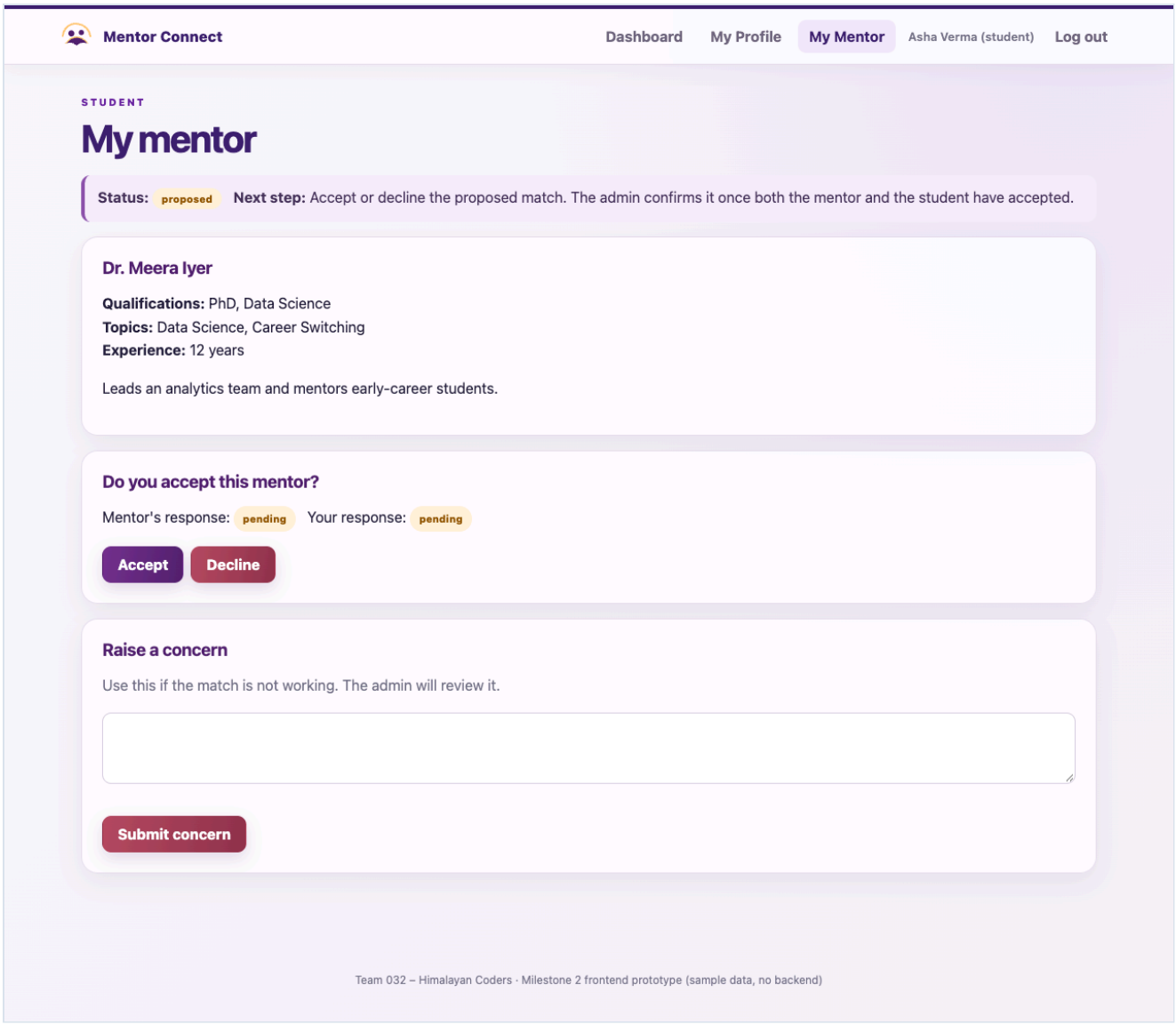


Figure 8. Student's mentor page while a match is proposed, with the Accept and Decline buttons.

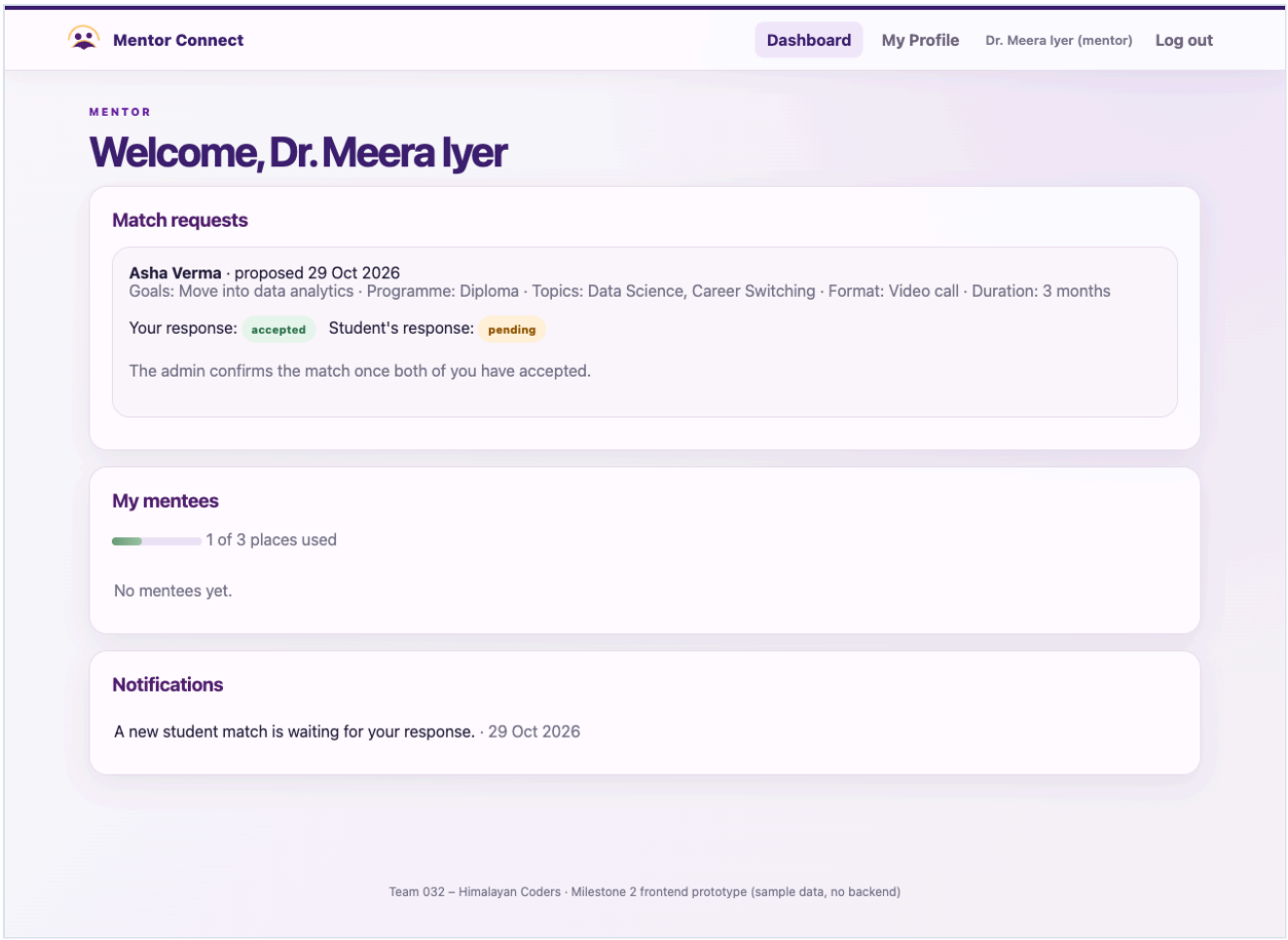


Figure 9. Mentor dashboard with a match request. It shows the mentor's and the student's answers.

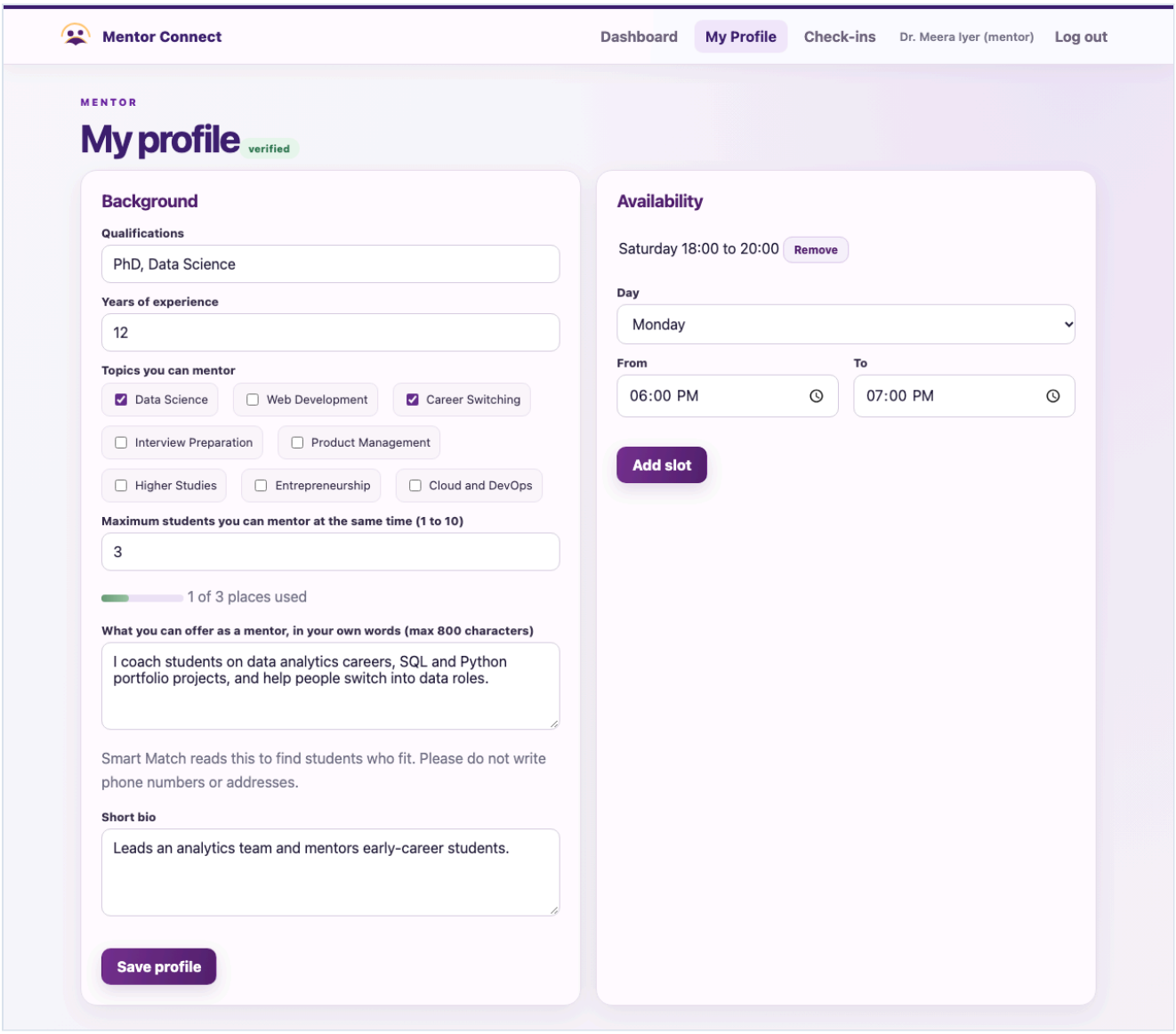


Figure 10. Mentor profile with topics, maximum students, the free-text box for Smart Match and availability slots.

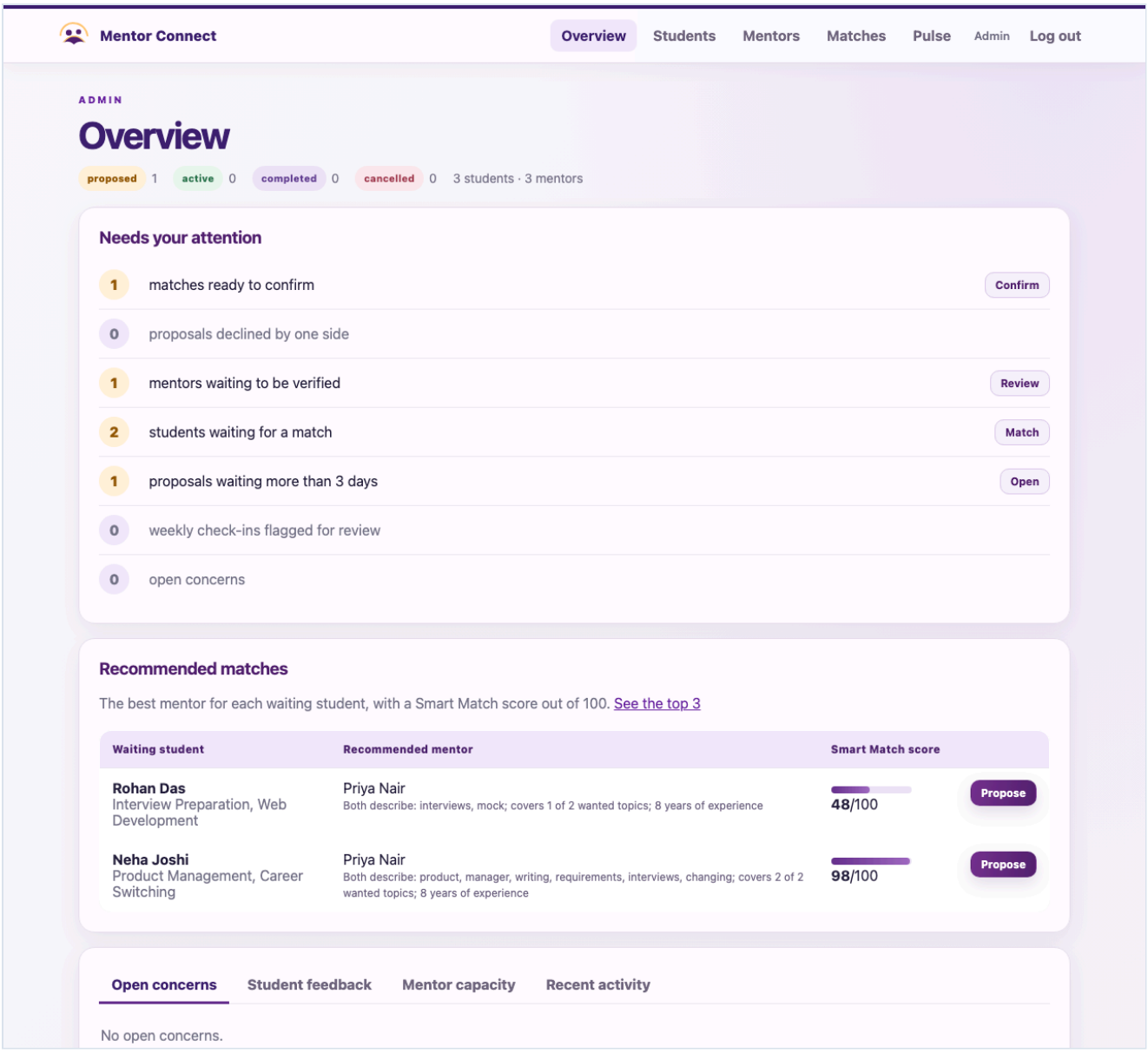


Figure 11. Admin overview: what needs attention (including flagged check-ins), the recommended matches with Smart Match scores, and tabs for concerns, feedback, capacity and activity.

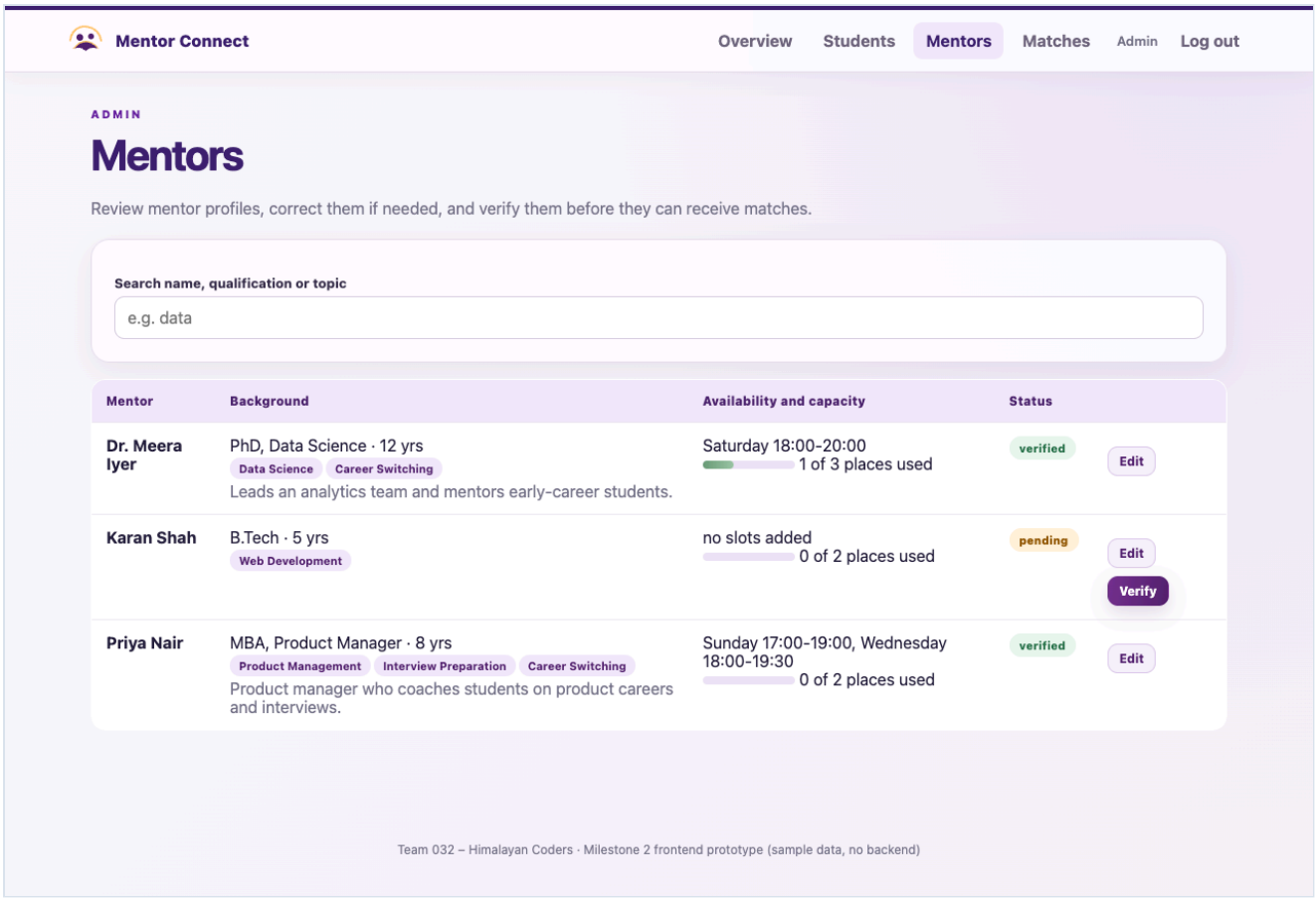


Figure 12. Mentor review with topics, availability and capacity.

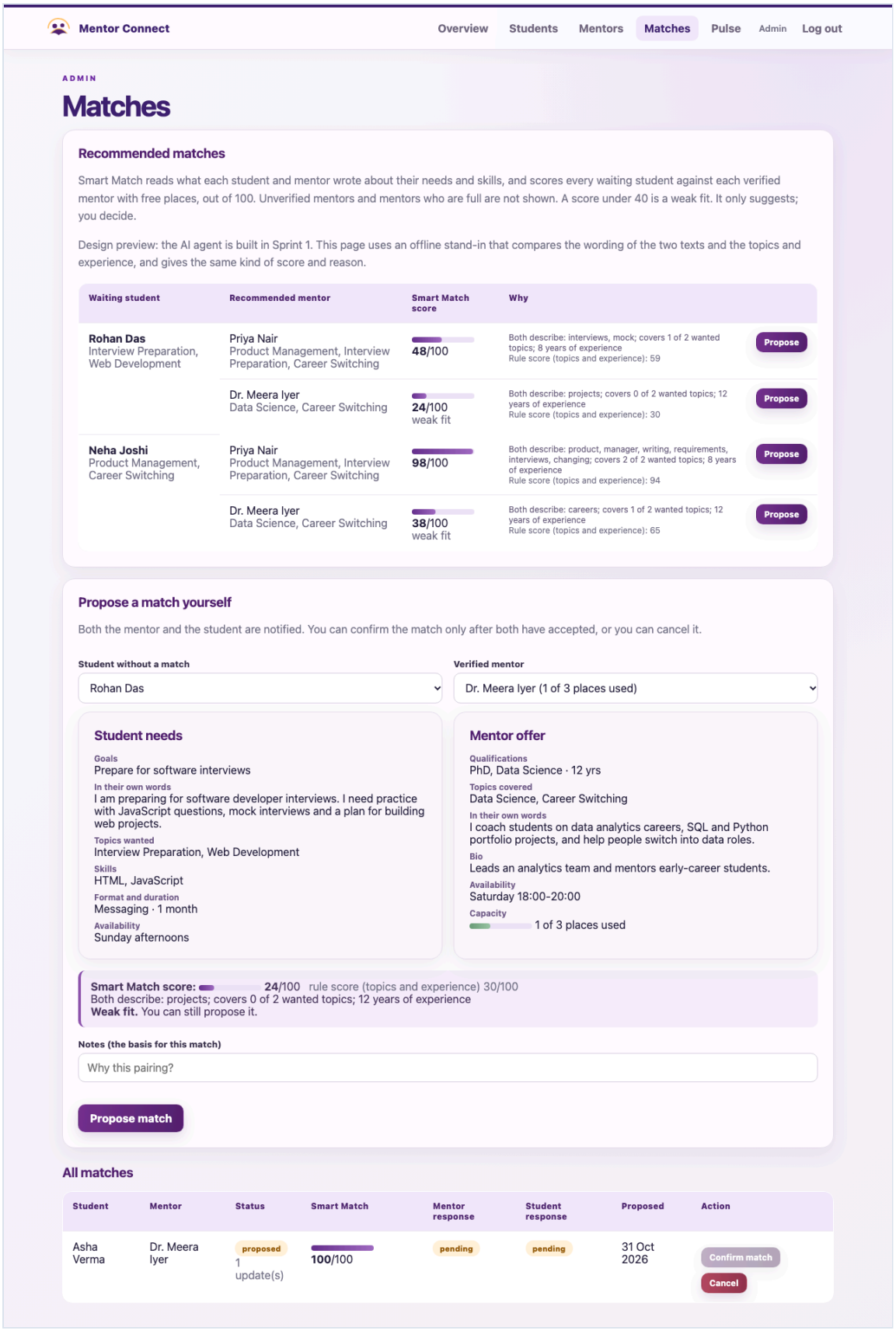


Figure 13. Matches page with the top three mentors for each waiting student, each with a Smart Match score and reason, the free text of a chosen pair, and the list of matches.

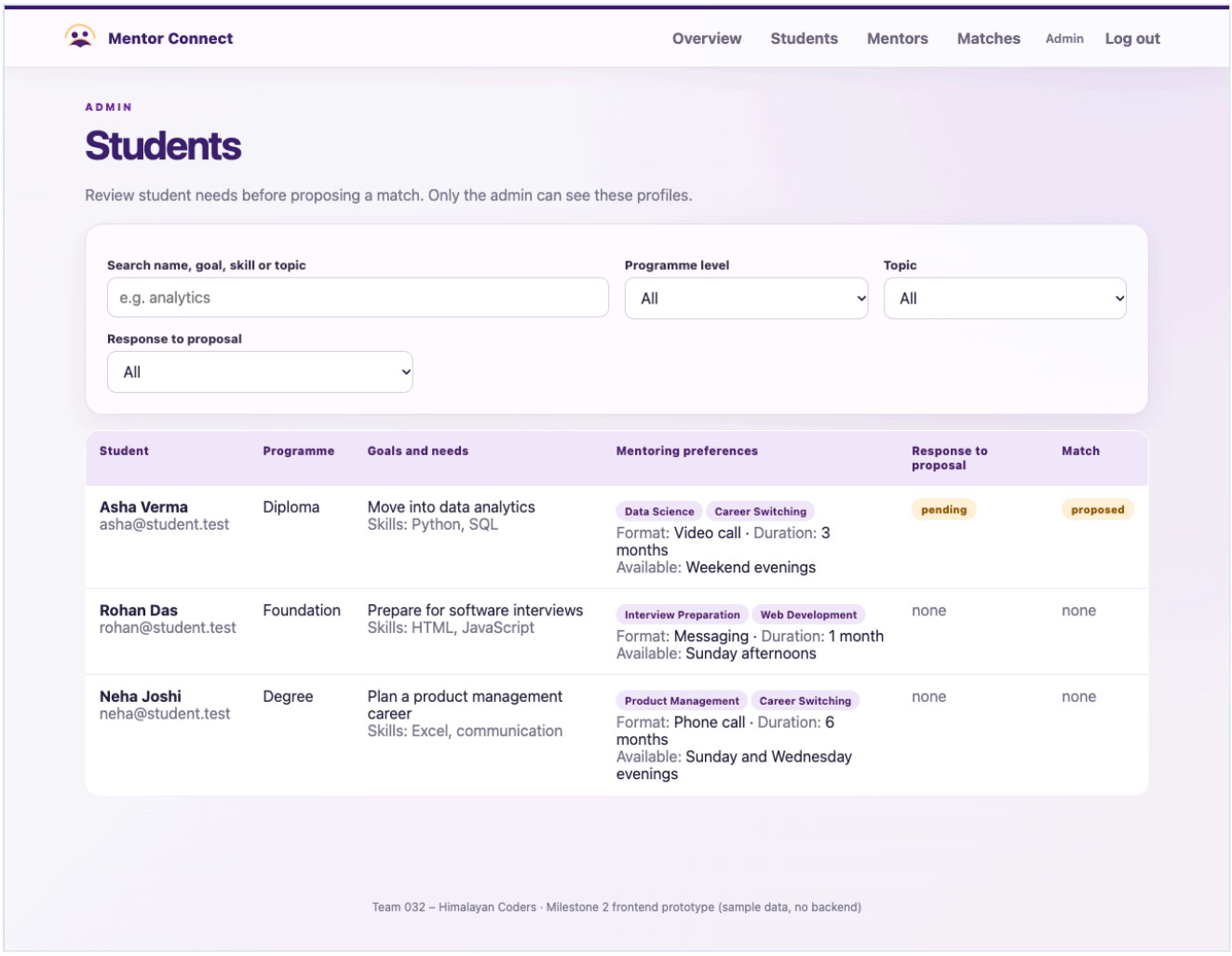


Figure 14. Student profiles with search and filters.

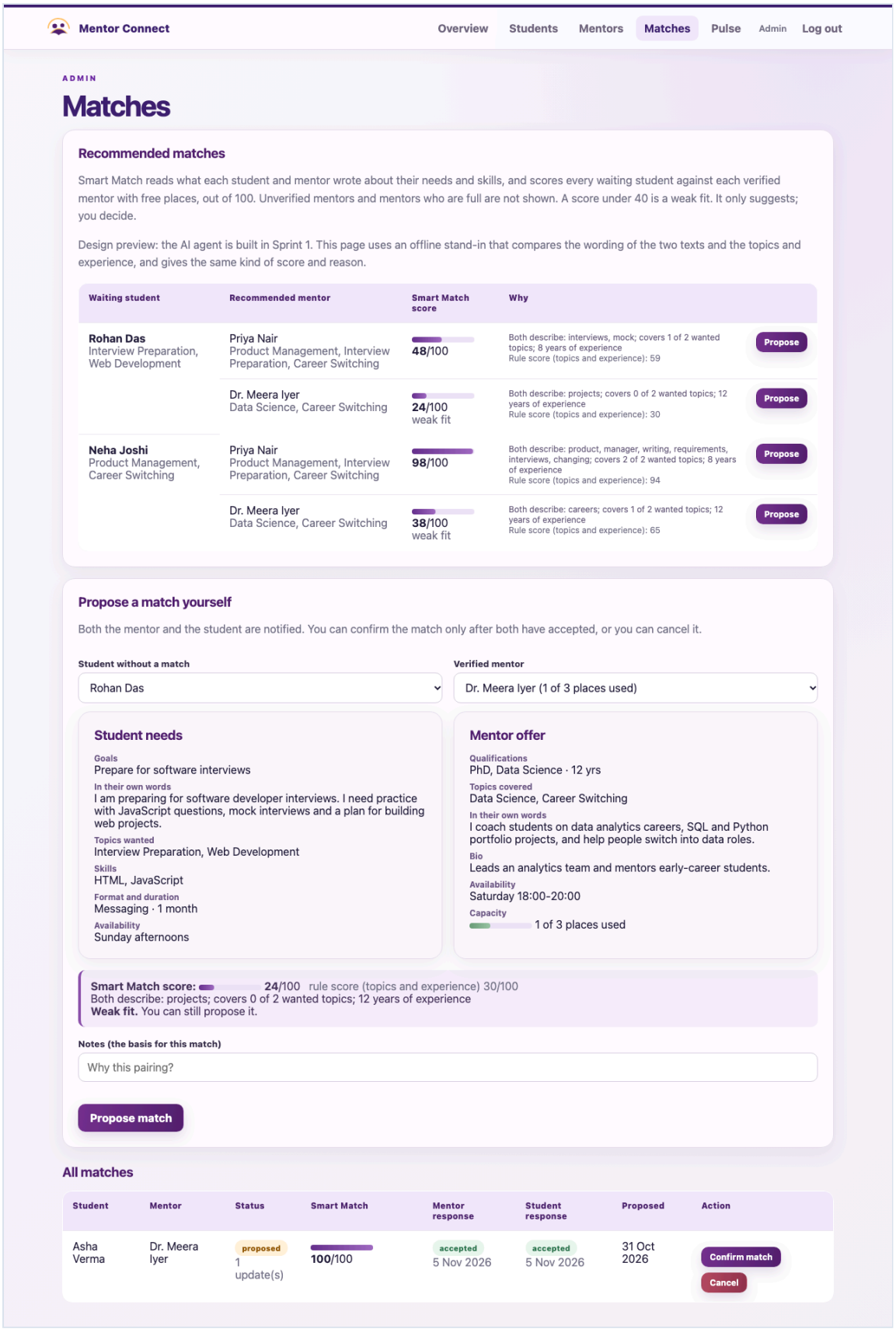


Figure 15. After both sides accept, the Confirm match button is enabled. Before that it is disabled.

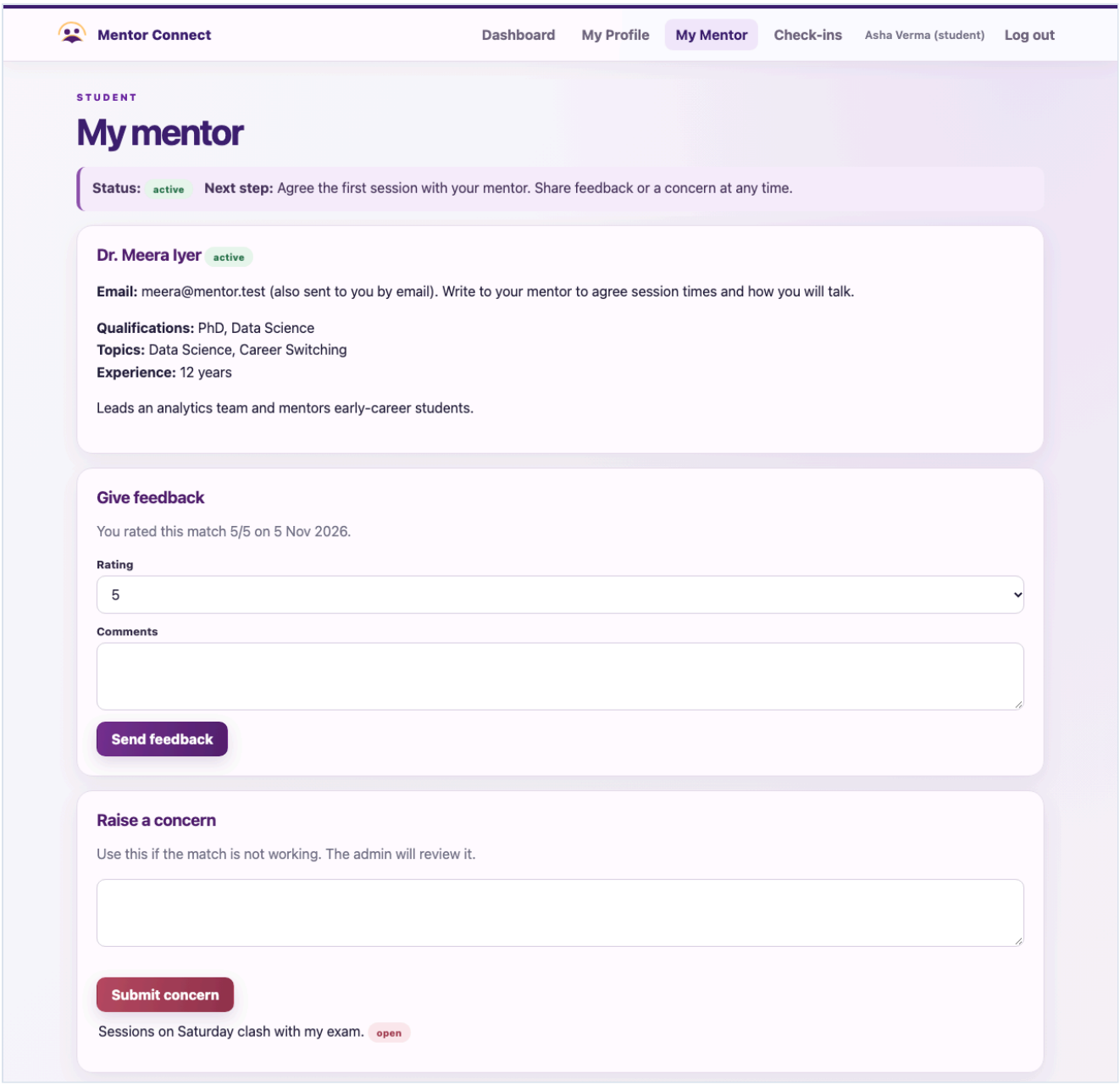


Figure 16. Student's mentor page after the admin confirms: status, next step, the mentor's email, feedback and concern forms.

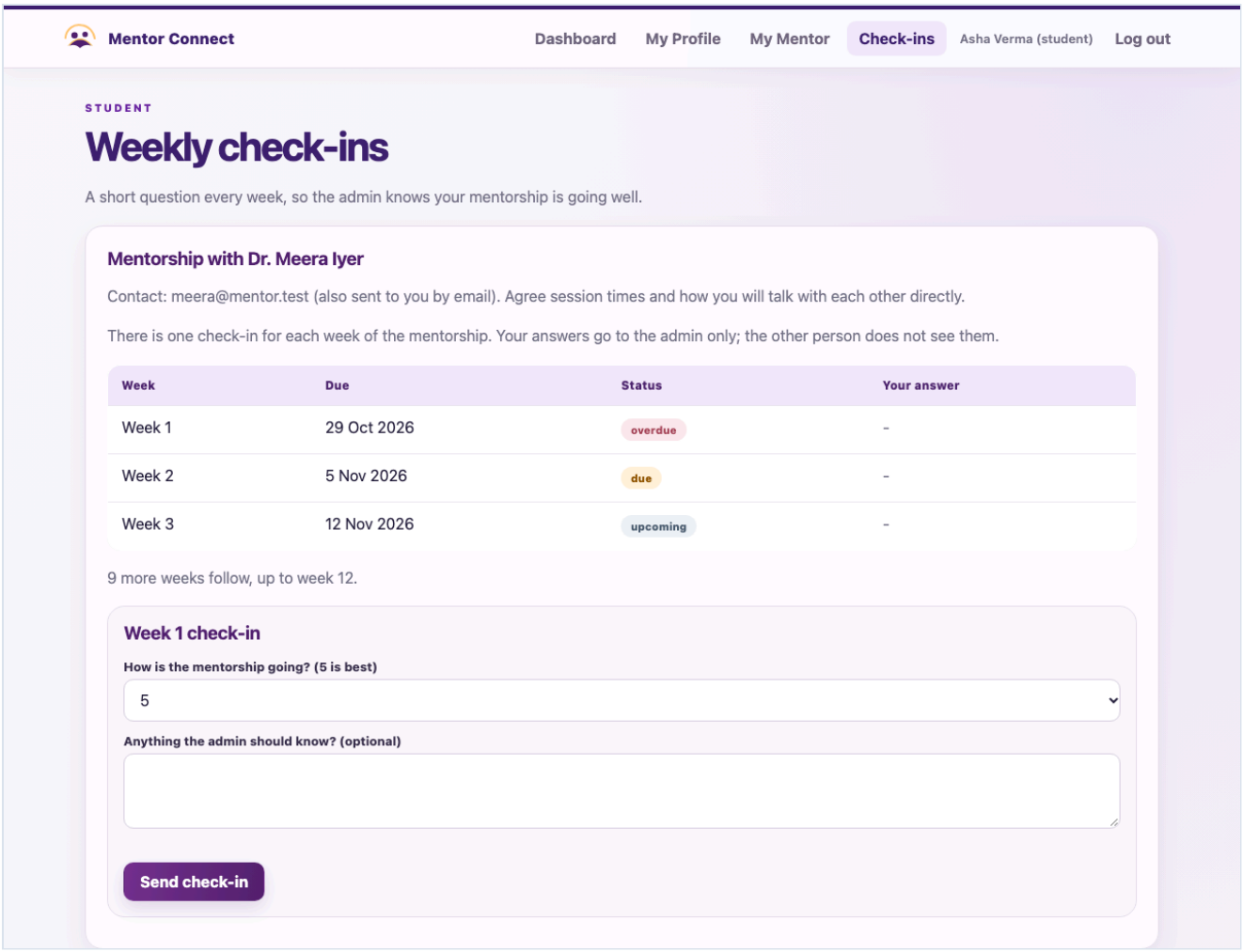


Figure 17. Student's weekly check-ins: the mentor's email, the state of each week, and the form for the week that is due.

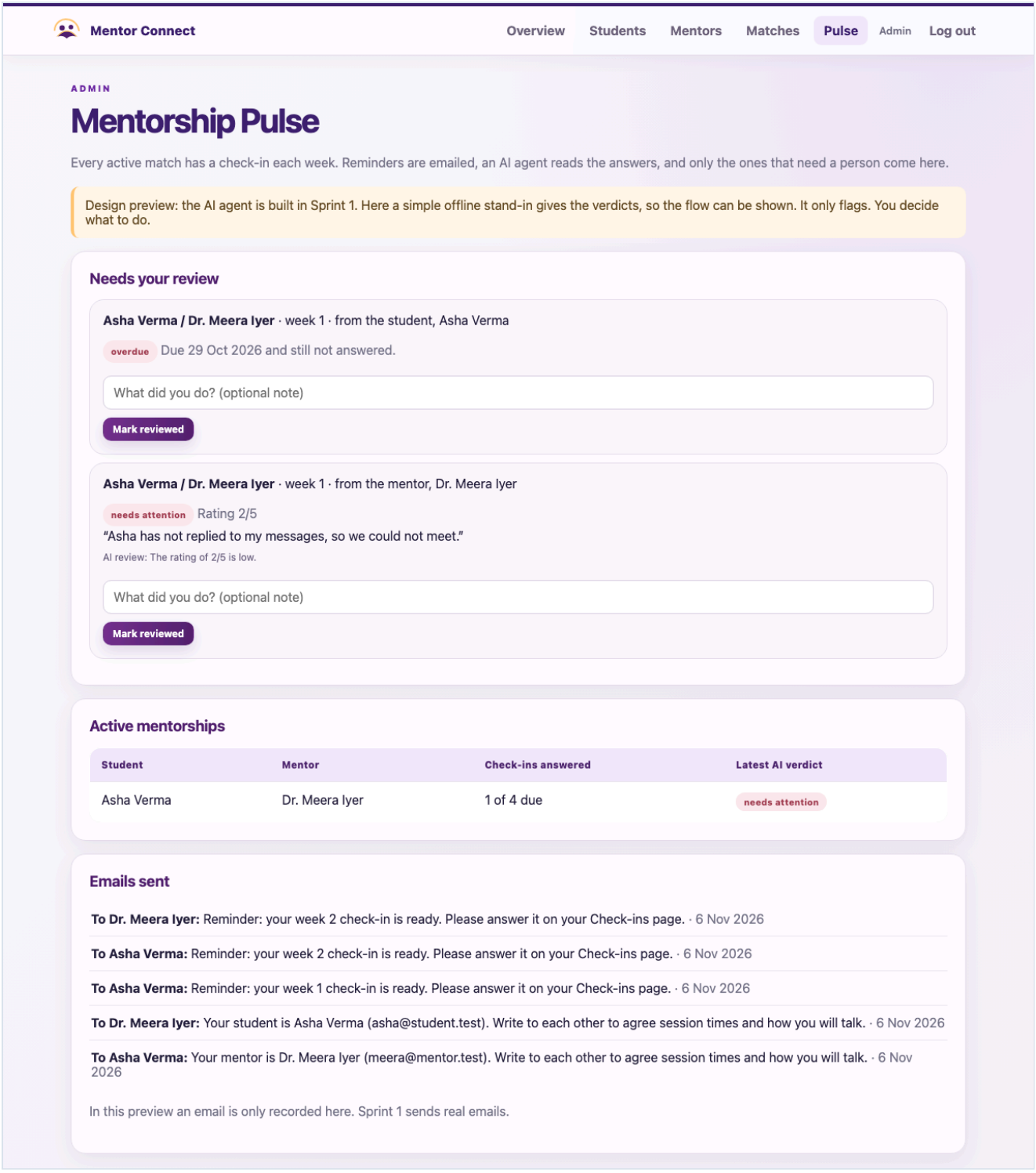


Figure 18. Mentorship Pulse for the admin: check-ins that need review with the AI's reason, active mentorships, and the emails sent. The AI verdicts in this prototype come from an offline stand-in.

8. Frontend Code and README

The frontend ZIP has the Vue project: the pages (`src/views`), shared components, the data layer (`src/services`), the styles, unit tests and a README. To run it you need Node.js 18 or newer. Unzip, open the `frontend` folder, then run `npm install` and `npm run dev`, and open the address shown. The password for the demo accounts (student, mentor and admin) is `demo123`. The accounts are listed on the login page. `npm run build` makes the files that a host such as Vercel serves, and `npm test` runs the unit tests.

Folder	What is in it
views	One file for each page, grouped as auth, student, mentor and admin.
components	Small shared parts: header, status badges, score and capacity bars, topic picker, recommendation table, comparison cards.
services	The data layer: login, matching and scores, Smart Match, match flow, profiles, weekly check-ins and their review, feedback, notifications. Replaced by API calls in Sprint 1.
router, layouts	Page addresses with the role guard, and the two page layouts.
config, data	Topics, statuses and score weights in one place, and the sample data.

9. Open Questions and Risks

- What should happen after a mentor or a student declines? In the prototype the admin decides, cancels the match and can propose a new one.
- Which AI provider and model do we use, who holds the key, and what is the monthly cost limit? The platform works without the agent through the fallback rules.
- Do students and mentors agree that the text they write is read by an AI service? The profile form says so; the client should confirm the wording.
- Which email service sends the contact and reminder emails, and which sender address is used?
- Is one check-in per week right for every duration, and is a grace period of 2 days acceptable?
- What should the admin do after a flag (call both people, change the match, cancel)? The prototype only records a note.
- How long is data kept, and who may see feedback and check-in answers? Today only the admin sees check-in answers.
- Should the system set a maximum number of students per mentor, and who keeps the list of topics?